
Moufida

(Arabic: “useful”)

The First 100% Local AI Diagnostic Platform for Entrepreneurs

Concept & System Presentation

Team Makrouna Kadheba

AINS 2026 Hackathon • June 2026

Abstract

Moufida is a voice-first, fully on-device AI companion that diagnoses, builds, and continuously monitors a founder’s startup. It is not a wrapper around a chat API: it is an analytical engine in which *every score decomposes to a formula, every claim traces to a real source, the entire AI stack runs locally via Ollama*, and a *Go daemon keeps the analysis alive* even when the application is closed. This document presents the full concept and system — the problem, the four architectural commitments, the polyglot microservice architecture (Python, Rust, Go, TypeScript), the two operating modes, the explainable scoring engine, the two interpretability research papers turned into production endpoints, the adaptive testing intake, the grounded RAG pipeline over 83 curated Tunisian-ecosystem resources, the continuous monitoring daemon, and the evaluation framework that backs the claims. Items that are aspirational or disclosed gaps are labelled explicitly.

Contents

1	What Moufida Is	3
1.1	Why It Is Different	3
2	The Problem	3
3	The Four Architectural Commitments	4
4	System Architecture	4
4.1	Component Map	4
4.2	Why Polyglot	5
5	The Orchestrator — Central Nervous System	5
5.1	Router Groups	5
5.2	Shared Modules	5

6	Two Operating Modes	5
6.1	Creation Mode (STATE_NEW) — Build From an Idea	6
6.2	Diagnosis Mode (STATE_EXISTING) — Know Where You Stand	6
7	Adaptive Intake — Computerized Adaptive Testing	6
7.1	The 2PL Model	6
7.2	Item Selection by Fisher Information	6
7.3	Bayesian EAP Ability Update	6
8	The Ten Axis Services — The Expert Panel	7
8.1	Three Dependency-Ordered Waves	7
8.2	Academic Grounding	7
9	Affinitree — The Deterministic Scoring Engine	8
9.1	The Formula	8
9.2	Evidence Tiers — The Quiet Innovation	8
9.3	Normalisation, Pre-Scoring, and Safety	8
9.4	Anomaly Detection	8
10	The Signal Service — Interpretability in Rust	9
10.1	Paper 1 — Label-Free Concept Bottleneck Model	9
10.2	Paper 2 — Contrastive Axis Direction Probe	9
10.3	Paper 3 — IRT/EAP Acceleration	9
11	Grounded Generation — RAG & Knowledge Base	9
11.1	Hybrid Retrieval Pipeline	9
11.2	Knowledge Base — 83 Curated Resources	10
12	The Go Daemon — The One That Never Sleeps	10
12.1	Watchers	10
13	Tool Integrations — Evidence That Counts	10
14	Advanced Analytical Modules	11
15	Liveness — Events & Real-Time Architecture	11
16	Voice, Local Models & the Desktop Companion	11
16.1	The 100%-Local Model Stack	12
16.2	Tauri Desktop App & Pixel-Art Companion	12
17	Persistence & Observability	12
17.1	Database	12
17.2	Admin Panel	12
18	Evaluation Framework	12

19 Honest State & Roadmap	13
19.1 Fully Implemented	13
19.2 Disclosed Gaps	13
19.3 The Path Forward	13
Selected References	13

1 What Moufida Is

“*Moufida*” is Arabic for “useful” — the name is a commitment. The product is a desktop AI that lives in the system tray. The founder describes a startup, by voice or text, in French, English, or Tunisian Arabic (Derja), and Moufida produces a complete diagnostic: a maturity assessment, five evidence-weighted composite scores, a named bottleneck per axis, a ranked blocker list, and a personalised roadmap linking real Tunisian support programmes. It then keeps watching the market, competitors, grants, and the regulatory environment in the background.

The one-sentence thesis

Grounded, private, explainable, and *alive* — a combination no mainstream founder tool offers today, delivered entirely on the founder’s own machine.

1.1 Why It Is Different

Typical AI business tool	Moufida
LLM improvises advice from training data	Every claim traced to a KB resource or an academic formula
A single vague score	Five composite scores, each decomposed into named, weighted sub-dimensions
Fixed 30+ question form	Adaptive intake converging in 8–15 questions
Advice for US/EU startups	83 curated resources specific to the Tunisian ecosystem
One-time snapshot	A 24/7 daemon watching competitors, grants, and regulation
Cloud-dependent	100% local: Ollama, Whisper, Piper — no data leaves the machine
Black-box reasoning	A Concept Bottleneck layer names the exact concept holding each score back

2 The Problem

Founders — especially in emerging ecosystems — are underserved on four axes at once:

- **Ungrounded advice.** Generic AI tools give confident answers with no traceability and no recourse.
- **Privacy cost.** Cloud SaaS advisors require sending strategy, financials, and intellectual property off the founder’s machine.
- **Black-box scores.** Typical ratings expose no formula, no evidence, and no way to challenge them.

- **Language and locality mismatch.** The real Tunisian ecosystem runs on French + Derja, and on local programmes (APII, Smart Capital, BFPME, StartupAct) that generic tools know nothing about.

These are treated not as features to bolt on later but as *architectural constraints* that shaped every design decision below.

3 The Four Architectural Commitments

Private by construction.

Diagnosis, scoring, and retrieval run entirely on-device. The single, documented exception is the *optional* Composio integration (managed OAuth), off by default.

Traceable by construction.

Every generated claim carries an EvidenceTrace; every roadmap action links to a verified URL; every score reduces to $c_i = w_i \times v_i \times m_i$.

Explainable by construction.

A Concept Bottleneck layer makes *which* concept blocks each score a first-class output.

Alive by construction.

A Go daemon and an event pipeline keep the analysis current with no user action.

4 System Architecture

Moufida is a polyglot microservice system deployed in two layers: native host processes (for GPU access and OS audio) and a Docker Compose stack (for everything else). Roughly fifteen coordinated services cooperate; the desktop app talks only to the orchestrator over HTTP and Server-Sent Events (SSE).

All LLM inference from inside Docker routes to the host Ollama at `host.docker.internal:11434`. No model weights ever live inside containers, and the datastores bind no host ports by default — there are no conflicts with a developer’s local Postgres or Redis.

4.1 Component Map

Component	Lang	Port	Role
Tauri Desktop App	React/TS + Rust	host	Primary UI, voice pipeline, companion window
Orchestrator	Python/FastAPI	8001	Central brain — 18 API routers
Axis Services (×10)	Python/FastAPI	8101–8110	Per-axis generate / diagnose / metric_update
Scoring Engine (Affinitree)	Python	8200	Deterministic formula-based scoring
RAG Service	Python/FastAPI	8300	Hybrid retrieval + KB mgmt + web search
Signal Service	Rust/Axum	8010	Concept Bottleneck + Axis Direction Probe
Go Daemon	Go	—	24/7 monitoring, 6 adaptive watchers
Admin Panel	React/Vite	3002	Read-only observability SPA
PostgreSQL	—	int.	All persistent relational data
Redis	—	int.	Pub/sub (daemon → orchestrator)
Qdrant	—	int.	Vector store for KB embeddings
SearXNG	—	int.	Self-hosted metasearch (no API keys)
Ollama	—	11434	Local LLM inference + embeddings

4.2 Why Polyglot

Language	Rationale
Python	Async fan-out and SSE streaming for orchestration and the ten domain services.
Rust	Microsecond numeric hot path — CBM weight layer, 9×1024 probe matmul, ridge regression. No GIL, static binary.
Go	A 24/7 supervisor with 6 goroutines uses <30 MB RAM; <code>context.Context</code> cancellation makes hot-swap race-free.
TypeScript + Rust (Tauri)	Native tray app, small footprint, host-local, with a Rust shell bridging to the voice binaries.

5 The Orchestrator — Central Nervous System

`backend/orchestrator/`, **port 8001**. A single FastAPI process owning 18 sub-routers, a background Redis consumer, and a telemetry middleware. Every action — diagnosis, plan generation, chat, pitch — passes through it.

5.1 Router Groups

- **Project lifecycle** — create / patch / list / delete projects (`state=NEW|EXISTING`, sector, language).
- **Adaptive intake** — a *stateless* CAT/IRT engine (§7).
- **Creation mode** — `/generate/{axis}` + `approve/retry` + `/finalize`.
- **Diagnostic mode** — the three-wave fan-out, CBM pass, anomaly detection, maturity staging; persists history, snapshots, and concept scores while SSE-streaming results.
- **Roadmap engine** — `regenerate` / `advance horizon` / `provenance`.
- **Advanced modules** — pitch simulator, persona simulator, pivot scenario planner (§14).
- **Intelligence** — competitor and opportunity ingestion from the daemon; watch-target refresh.
- **Events / history / chat** — the SSE stream, event act/ignore, diagnostic comparison, and grounded chat with LLM intent detection.
- **Daemon control plane** and the read-only **Admin API**.

5.2 Shared Modules

Module	Responsibility
<code>axis_registry.py</code>	Single source of truth: axis slugs, ports, score ownership, <code>DIAGNOSTIC WAVES</code> , signal-to-axis routes, tool-to-axis maps.
<code>dependency.py</code>	Directed dependency graph with transitive re-run resolution; resolves the business-model $\leftrightarrow$ operations cycle (the only SCC).
<code>sse.py</code>	Per-project in-memory <code>asyncio.Queue</code> per subscriber.
<code>redis_consumer.py</code>	Background task routing daemon signals to axis <code>metric_update</code> ; interprets <code>significant_change</code> .
<code>telemetry_middleware.py</code>	Records every request; seeds <code>request_id</code> / <code>project_id</code> ContextVars so all LLM calls correlate.
<code>evidence_snapshot.py</code>	<code>gather_evidence()</code> builds the snapshot used by pitch / persona / scenario.

6 Two Operating Modes

6.1 Creation Mode (STATE_NEW) — Build From an Idea

The founder types an idea (“an app that helps bakeries manage inventory”) and Moufida builds a nine-axis plan, one section at a time. Each generation call fetches fresh KB evidence and live SearXNG results, so every proposal is citation-backed. The founder **approves**, **edits with constraints** (replays generation with constraints appended), or **retries** (higher temperature). When all nine axes are approved, POST /finalize synthesises a three-horizon roadmap with full KB provenance recorded in roadmap_versions.

6.2 Diagnosis Mode (STATE_EXISTING) — Know Where You Stand

The founder describes an existing startup; the adaptive intake collects a profile; the three-wave engine scores it; the CBM pass names each bottleneck; anomaly detection flags contradictions; and a personalised roadmap is produced. Results stream to the dashboard progressively over SSE.

7 Adaptive Intake — Computerized Adaptive Testing

Rather than a fixed 30+ question form, Moufida selects each next question to maximise information about *this* founder, converging in 8–15 questions. The math is Item Response Theory (the same family used by the GMAT and GRE).

7.1 The 2PL Model

The probability that a founder at latent maturity θ answers item j positively:

$$P(u_j = 1 \mid \theta) = \frac{1}{1 + \exp(-a_j(\theta - b_j))},$$

where a_j is item *discrimination* (how sharply it separates stages) and b_j is item *difficulty* (the stage at which a positive answer becomes likely).

7.2 Item Selection by Fisher Information

The information item j provides at ability θ is

$$I_j(\theta) = a_j^2 P_j(\theta) (1 - P_j(\theta)),$$

maximised when difficulty matches ability ($\theta \approx b_j$) and discrimination is high. The engine always asks $j = \arg \max_j I_j(\hat{\theta})$ among unasked items.

7.3 Bayesian EAP Ability Update

After each response, the ability estimate is the Expected A Posteriori value

$$\hat{\theta} = \frac{\int \theta L(\theta \mid u_{1:n}) p(\theta) d\theta}{\int L(\theta \mid u_{1:n}) p(\theta) d\theta},$$

computed over a 33-point quadrature grid with a Gaussian prior. The posterior standard error $SE(\hat{\theta})$ gives a stopping rule: **stop when** $SE(\hat{\theta}) < 0.40$ **or** ≥ 12 **items answered**. Graded (yes / partially / not yet) responses use the Samejima Graded Response Model extension.

Architecturally unusual: a stateless CAT server

On every POST /intake/next, the client replays the full answer history and the server re-derives $\hat{\theta}$ from scratch. No session state, no per-question DB read — horizontally scalable, trivially

retryable. A Rust `/cat/eap` endpoint accelerates the EAP; a pure-Python fallback means it is never blocking.

8 The Ten Axis Services — The Expert Panel

`backend/services/`, **ports 8101–8110**. Ten domain microservices, each exposing an identical contract: `/generate` (creation), `/diagnose` (diagnosis), and `/metric.update` (receive daemon signals). Each `/diagnose` call also runs `run_due_diligence()` for investor-grade flags (SAM/SOM derivation, LTV/CAC, IP ownership chain, key-person risk).

#	Axis	Port	Assesses
1	Ideation	8101	Problem clarity, originality, founder-market fit; also classifies maturity
2	Market Intelligence	8102	TAM/SAM/SOM, customer validation, revenue model, competition
3	Product Offering	8103	MVP existence, feasibility, feature definition, UX
4	Brand & Innovation	8104	Novelty: product/tech 35%, market 25%, brand 20%, value 20%
5	Business Model	8105	Revenue model, unit economics, pricing, path to profit
6	Legal Compliance	8106	GDPR, AI Act, IP, registration, SDG alignment
7	Marketing	8107	GTM strategy, channels, messaging, content presence
8	Sales	8108	Pipeline, sales process, conversion, traction
9	Operations	8109	Team, process, suppliers, operational capacity
10	Go-to-Market	8110	Launch-readiness synthesis; roadmap integration

8.1 Three Dependency-Ordered Waves

```
Wave 0 (parallel): ideation | market | legal v (depends on wave 0 scores) Wave 1
(parallel): product | business-model | operations v (depends on wave 1 scores) Wave 2
(parallel): brand | marketing | sales | go-to-market
```

Axes within a wave run concurrently via `asyncio.gather`; no service touches go-to-market before business model and operations have spoken. Total diagnosis time $\approx 3\times$ the slowest single-axis call.

8.2 Academic Grounding

Composite score	Owner axis	Key references
Market	market-intelligence	Kotler & Keller (2016); Blank (2013)
Commercial Offer	product-offering	Osterwalder et al. (2014)
Innovation	brand-innovation	OECD Oslo Manual (2018)
Scalability	business-model + operations	Ries (2011); Christensen (1997)
Green	legal-compliance	EU AI Act; GDPR; UN SDG framework

9 Affinitree — The Deterministic Scoring Engine

backend/scoring-engine/, **port 8200** (also used in-process by the axis services). Every formula, weight, and normaliser lives in JSON config with inline academic citations. No randomness, no neural network, no floating-point non-determinism.

9.1 The Formula

Each sub-dimension’s contribution is

$$c_i = w_i \times v_i \times m_i,$$

and the composite score is $\sum_i c_i$ rescaled to $[0, 5]$, where w_i is the configured weight, v_i the normalised field value, and m_i the *evidence-tier multiplier*.

9.2 Evidence Tiers — The Quiet Innovation

Tier	m_i	Source
T1 — Declared	0.6×	Founder self-reported in intake
T2 — Artefact-backed	1.0×	Uploaded document, Notion page
T3 — Daemon-observed	1.2×	GitHub commits, Google Analytics, competitor monitor

The same field value scores higher when backed by real evidence: a founder whose GitHub shows 200 commits last month (T3) out-scores one who merely *claims* strong engineering (T1). Connecting a tool can lift `product.mvp.existence` from T1 to T3 — a 100% increase in that contribution. Evidence quality is a first-class term in the model.

9.3 Normalisation, Pre-Scoring, and Safety

- **Normalisers:** `saturating(max)` (plateauing values), `ordinal_map` (discrete categories), `inverse_log_density` (more competitors → lower score).
- **Text pre-scoring:** free-text fields (value proposition, differentiation) are scored 0–4 against a structured rubric by Ollama at temperature 0.1, cached in `profile.rubric_scores`, and fed as v_i . This is the *only* non-deterministic component.
- **Safety:** the formula evaluator uses AST parsing with a strict arithmetic whitelist — no `eval`. Config files are safe to edit.

9.4 Anomaly Detection

Ten rule-based contradiction detectors run after scoring, e.g. `revenue_without_interviews`, `negative_unit_economics` ($LTV < CAC$), `critical_runway` (<3 months), `ai_act_noncompliance`, `high_burn_no_revenue`, `ip_gap`, `market_size_mismatch`. Evaluation target: 100% recall across 10 fixture profiles — verified.

Why determinism matters

Run the same profile twice and the scores are bit-identical. This is what makes *Score Debate* meaningful: when a human expert challenges a score in chat and an LLM judge is convinced, the row is locked (`locked=true`) and never overwritten by an automatic re-run. The baseline never drifts on its own, so an override means something.

10 The Signal Service — Interpretability in Rust

signal/, port 8010 (**Rust + Axum**). A stateless microservice with no DB and no LLM calls, turning two research papers into production endpoints. It is *best-effort*: if it is down, diagnostics still complete (CBM cards simply vanish) and RAG falls back to standard rank fusion.

10.1 Paper 1 — Label-Free Concept Bottleneck Model

Koh et al. (ICML 2020); Oikarinen et al., Label-Free (ICLR 2023). Instead of `profile` $\rightarrow$ `score`, the prediction must pass through named, human-readable concepts: `market` $\rightarrow$ {TAM evidence, ICP specificity, WTP signal, ...}. The LLM scores each concept $c_i \in [0, 1]$; a small, auditable linear head produces the score:

$$\text{axis_score} = \sigma(w^\top c + b) \times 5.0.$$

The **bottleneck** is the concept with the highest improvement potential,

$$\text{bottleneck} = \arg \max_i [w_i (1 - c_i)],$$

and the projected lift of fixing concept i to a target c'_i is a concrete number the roadmap can act on (“raising ICP specificity from 0.18 to 0.60 moves the market score from 2.3 to 3.1”). Weights start from domain priors in `concepts.json` and, once ≥ 20 snapshots per axis accumulate, are recalibrated by ridge regression

$$w = (C^\top C + \lambda I)^{-1} C^\top y$$

in microseconds (`nalgebra`, $k \leq 5$). The UI shows a “prior / calibrated” chip per axis.

10.2 Paper 2 — Contrastive Axis Direction Probe

Zou et al., Representation Engineering (2023); Park et al., Linear Representation Hypothesis (2023). Concepts are *directions* in embedding space. Nine unit vectors — one per axis — are computed offline as the normalised contrastive mean

$$d_{\text{axis}} = \frac{\mu_{\text{pos}} - \mu_{\text{neg}}}{\|\mu_{\text{pos}} - \mu_{\text{neg}}\|}$$

over `bge-m3` embeddings. A 9×1024 matrix-vector product $D^\top e$ (in $< 5 \mu\text{s}$) scores any chunk’s relevance to all nine axes. Two uses: (1) **RAG re-ranking** — blend 30% axis relevance into the fusion score; (2) **auto-tagging** every newly ingested document with its top-2 axes.

10.3 Paper 3 — IRT/EAP Acceleration

`POST /cat/eap` provides the Rust-accelerated Bayesian EAP for the intake engine (§7), with the Python fallback ensuring it is never on the critical path.

11 Grounded Generation — RAG & Knowledge Base

`backend/rag/`, port 8300. Grounding is a *retrieval contract*, not a prompt instruction.

11.1 Hybrid Retrieval Pipeline

1. **Dense** — embed the query via `bge-m3` (1024-dim); Qdrant cosine search for the top-40.
2. **Sparse** — BM25 over the candidate pool text.
3. **Reciprocal Rank Fusion** ($k=60$): parameter-free, robust to the scale gap between cosine and BM25 (Cormack et al. 2009).
4. **Sector boost** — matching-sector chunks $\times 1.3$.

5. **Axis-direction re-rank** — when a `current_axis` is given,

$$\text{final} = 0.7 \text{ rrf} + 0.3 \text{ axis_relevance}.$$

The axis re-rank fixes a real failure: a “BTS micro-credit” chunk matches a market query on keywords yet belongs to business-model — the axis signal demotes it for the market axis while keeping it for business-model.

11.2 Knowledge Base — 83 Curated Resources

65 base Tunisian-ecosystem resources + 18 multilingual per-axis resources, each tagged with stage, sector, type, and score dimensions. Institutions covered include **APII** (StartupAct certification), **Smart Capital**, **BFPME**, **BTS**, **INNORPI**, **CEPEX**, and **Flat6Labs / Orange Digital Center**. Because `bge-m3` is multilingual, a Derja voice query retrieves French-language resources from the same semantic space. Live web evidence comes from self-hosted **SearXNG** — no API key, no third-party query exposure. The daemon’s staleness watcher flags resources with `last_verified` > 90 days.

12 The Go Daemon — The One That Never Sleeps

`daemon/` (**Go 1.22**). Outbound-only, no HTTP server. It boots without knowing what to watch; it learns by polling `GET /daemon/control` (a single-row table carrying `paused` and `focus_project_id`).

Hot-swap without restart. On a focus change the supervisor cancels the current watcher batch (`context.CancelFunc` + a `sync.WaitGroup` drain), refreshes LLM-derived watch targets (cached by profile hash), and starts a fresh batch — all in-process. Pause stops the work but keeps goroutines alive so the UI can distinguish “paused” from “offline.”

12.1 Watchers

Watcher	Cadence	Signal → axis / surface
Budget	6 h	<code>budget.alert</code> → business-model
Competitor	12 h	<code>page.diff</code> → LLM SWOT → Competitor Board
Legal	24 h	<code>legal.update</code> → legal
Milestone	24 h	<code>milestone</code> → ideation
Trend	7 d	<code>trend.spike</code> → market
Grant	24 h	<code>match.scored</code> → Opportunity Radar
KB Staleness	nightly	<code>needs.review</code> (process-lifetime)
Composio Poll	5 min	NAT-friendly inbound-trigger fallback (process-lifetime)

Signals reach the orchestrator over Redis pub/sub (`moufida:signals`) or, for rich observations, HTTP `POST /competitor/observe` and `/opportunity/observe`. The pixel-art companion is the literal on/off switch: a sleeping character means monitoring is paused.

13 Tool Integrations — Evidence That Counts

A `ToolManager` facade is called twice per diagnostic: `enrich_profile()` before scoring (pull tools upgrade evidence tiers — never downgrade) and `dispatch_diagnostic()` after (push tools post results).

Tool	Type	Effect
GitHub	Pull	<code>ops.engineering_velocity</code> , <code>product.mvp_existence</code> → T3
Notion	Pull	product / ideation / market fields → T2/T3
Google Sheets	Pull + Push	finance fields → T2/T3; pushes score rows
Google Analytics	Pull	customer count, conversion, channels → T3
Slack	Push	posts briefings and score alerts
Composio	Bidirectional	managed-OAuth gateway + inbound triggers

The one deliberate trade-off

The analytical pipeline is 100% local. The single exception is Composio’s hosted OAuth popup and trigger broker — a conscious, documented choice for no-credential, bidirectional sync. Leave `COMPOSIO_API_KEY` empty and those tools report “unavailable” while hand-rolled token tools keep working. We name our one exception rather than quietly phoning home.

14 Advanced Analytical Modules

All three are grounded *only* in the founder’s own diagnostic data, each claim wrapped in an `EvidenceTrace`.

- **Investor Pitch Simulator** — spar with a Seed VC / Angel / Impact / Strategic persona whose questions come only from the project’s real scores, blockers, competitors, and opportunities. Ends with a per-axis `PitchReadinessReport` and prep actions pushable to the roadmap.
- **Pivot Scenario Planner** — up to three “what-if” profile overrides → projected score delta on all nine axes with confidence and cited reasoning. Adopt → patch profile → re-diagnose.
- **Customer Persona Simulator** — 3–5 evidence-grounded Tunisian personas (archetype, region, budget, objections, buying triggers) you can chat with to surface objections.

15 Liveness — Events & Real-Time Architecture

After the first diagnosis, four update sources feed one shared pipeline:

Source	Example	Entry point
Manual edit	Founder edits a plan section	<code>POST /event source=manual</code>
Chat intent	“We pivoted to B2B”	LLM intent → <code>apply_update()</code>
Tool signal	GitHub push via Composio	<code>tool_signals</code> → <code>dispatch</code>
Daemon event	Competitor shipped a feature	Redis → <code>redis_consumer.py</code>

Each becomes an **Event Card** (field-level diff). The founder chooses *Apply* / *Handle myself* / *Ignore*; the dependency engine re-runs only affected axes (a market signal cascades to product → brand → marketing → sales, but not to legal); results stream back over SSE. The dedicated `/events/stream` path carries ~14 event types (`score_update`, `alert`, `roadmap_update`, `concept_update`, ...) that also drive the companion’s reactions.

16 Voice, Local Models & the Desktop Companion

16.1 The 100%-Local Model Stack

Model	Role	Where
llama3.1:8b (~4.7 GB)	Generation, chat, pitch, debate	Ollama (host)
bge-m3 (~1.2 GB)	Multilingual 1024-dim embeddings	Ollama (host)
Whisper large-v2 Q5	STT (FR / EN / Derja)	host binary
Piper FR + Kokoro-82M	TTS	host ONNX
fastText lid.176	Per-utterance language ID	WASM, <1 ms

A full three-wave diagnosis runs in ~15–25 s on GPU, ~60–120 s on CPU. Founder data is embedded, reasoned over, transcribed, and spoken back entirely on-device — it never reaches an external server.

16.2 Tauri Desktop App & Pixel-Art Companion

A Tauri v2 app (React 18 + TypeScript in a Rust shell) lives in the system tray with two windows. The Rust shell exposes seven commands (transcribe, speak, read dropped file, open URL, window control). State is a flat Zustand store; the SSE consumer dispatches 13 event types to both the store and the companion. The companion is pure Canvas 2D (no sprites) with **24 expressive states** and 5 palette themes: she celebrates milestones, looks worried on a score drop ≥ 1.0 , sleeps when the daemon is paused, and ingests PDFs dropped onto her. The UI has full FR/EN/AR parity (~260 keys each) with automatic RTL.

17 Persistence & Observability

17.1 Database

PostgreSQL 16 with 20 idempotent migrations. `profiles` is the root aggregate; every table is project-scoped. Notable design: `score_snapshots` carries the `locked` flag protecting human overrides; `roadmap_versions` records a `kb_version` (SHA-256 of KB state) for stale-roadmap detection; `daemon_control` is a single row with `focus_project_id` ON DELETE SET NULL so deleting a focused project parks the daemon gracefully. Migration 020 adds three telemetry tables (`api_requests`, `llm_calls`, `daemon_activities`); only a 280-char preview and a SHA-256 of each prompt are stored.

17.2 Admin Panel

A read-only React SPA (port 3002) over `/api/admin/*`: live service-health probes; a paginated request log with click-through **distributed traces** from one HTTP request to all its correlated LLM calls (model, token counts, previews); a daemon-activity log; and a live colour-coded SSE log stream. Judges can watch a diagnosis run end-to-end in real time — no black-box trust.

18 Evaluation Framework

Tier	Subsystem	Metric	Status
T2a	Scoring determinism	10-run identity = 100%	Pass
T2c	Anomaly recall	10/10 contradictions	Pass
T2b	Rubric LLM stability	$\sigma \leq 0.15$ / field	needs Ollama
T1	Maturity classifier	Macro-F1 ≥ 0.65 , Top-2 ≥ 0.85	Phase 2
T3	RAG retrieval	Recall@3 ≥ 0.80 , MRR ≥ 0.70	Phase 3

The dependency engine has 12 passing unit tests, including the business-model $\leftrightarrow$ operations SCC. End-to-end smoke tests and `run-all-evals.sh` tie the tiers together.

19 Honest State & Roadmap

19.1 Fully Implemented

All five explainable scores with breakdown trees; nine axis `/diagnose+generate` endpoints; the dependency engine; adaptive intake; hybrid RAG + web search; the 24/7 Go daemon with hot-swap watchers; the Rust interpretability service; observability telemetry; the full creation/diagnosis frontend; and FR/EN/AR i18n.

19.2 Disclosed Gaps

We ship the gap list with the demo: the LangGraph `StateGraph` is not yet compiled (the `TypedDict` exists); the app icon is a placeholder; the Porcupine wake word needs a free Picovoice `.ppn`; and Axis-10 `/diagnose` is a stub (the roadmap is handled by an orchestrator RAG call).

19.3 The Path Forward

Near term: compile the `StateGraph`, harden Axis-10, ship signed builds, and validate the T1/T3 targets at scale. Then: grow the KB, deepen Derja support, and move from a single-founder tool toward the shared diagnostic backbone that incubators and accelerators standardise on — re-pointing the engine at new markets by swapping data, not code. See `roadmap.md` for the full plan.

Why this architecture wins

Traceable, private, alive, and explainable *by construction* — and polyglot, decoupled, and gracefully degrading. The hard parts (local multi-service orchestration, explainable scoring, interpretability, a self-updating daemon) already work. The roadmap is about scale and polish, not net-new invention.

Selected References

- Koh, P. W., et al. (2020). Concept Bottleneck Models. *ICML 2020*. arXiv:2007.04612.
- Oikarinen, T., et al. (2023). Label-Free Concept Bottleneck Models. *ICLR 2023*. arXiv:2304.06129.
- Zou, A., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.
- Park, K., et al. (2023). The Linear Representation Hypothesis and the Geometry of LLMs. arXiv:2311.03658.
- Lord, F. M. (1977). Practical applications of item characteristic curve theory. *J. Educ. Meas.* 14(2).
- Bock, R. D., & Mislevy, R. J. (1982). Adaptive EAP estimation of ability. *Appl. Psych. Meas.* 6(4).
- Samejima, F. (1997). The graded response model. In *Handbook of Modern IRT*. Springer.
- Cormack, G. V., et al. (2009). Reciprocal Rank Fusion. *SIGIR 2009*.
- Osterwalder, A., et al. (2014). *Value Proposition Design*. Wiley.
- Ries, E. (2011). *The Lean Startup*. Crown Business.

“Moufida” means useful. Everything above is in service of that — nothing ships that does not make the companion more useful, more trustworthy, or more alive.